

Федеральное государственное автономное образовательное учреждение
высшего образования

**«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»**

Отчет

по лабораторной работе №2.2

«Двоичные деревья поиска и AVL-деревья»

по дисциплине «Алгоритмы и структуры данных»

Автор: Бен Шамех Абделаиз

Факультет: ПИН

Группа: K3239

Преподаватель: Ромакина Оксана Михайловна

Санкт-Петербург 2026

1. Цель работы

Цель лабораторной работы — освоение структуры данных «двоичное дерево поиска» (BST) и её сбалансированной версии АВЛ-дерево. Реализованы проверка корректности BST, удаление поддеревьев, левый поворот АВЛ, вставка в АВЛ и k-й максимум.

2. Задание на лабораторную работу (Вариант 26)

- Задача 6. Оpozнание двоичного дерева поиска — вывод CORRECT / INCORRECT. [1.5 балла]
- Задача 9. Удаление поддеревьев — после каждого удаления вывести число вершин. [2 балла]
- Задача 13. Делаю я левый поворот — выполнить левый поворот АВЛ-дерева. [3 балла]
- Задача 14*. Вставка в АВЛ-дерево с балансировкой (доп.). [3 балла]
- Задача 16*. K-й максимум — структура данных с add/remove/kth_max. (доп.). [3 балла]

3. Краткие теоретические сведения

Двоичное дерево поиска (BST): для каждого узла V все ключи левого поддерева $< \text{key}(V)$, правого $> \text{key}(V)$. Проверка — DFS с поддержанием границ (lo, hi) .

АВЛ-дерево — BST с инвариантом: $|h(\text{left}) - h(\text{right})| \leq 1$ для каждого узла. Балансировка через малые/большие левые и правые повороты. Гарантирует $h = O(\log n)$ и $O(\log n)$ для всех операций.

4. Выполнение работы

4.1. Задача 6 — Оpozнание BST

Рекурсивный DFS: для каждого узла i с ключом K_i проверяем $lo < K_i < hi$. Корень получает границы $(-\infty, +\infty)$. Левый ребёнок наследует (lo, K_i) , правый — (K_i, hi) . Пустое дерево — CORRECT. Все ключи различны.

Листинг 1 — task6.py

```
def is_bst(tree, node_idx, min_val, max_val):
    if node_idx == -1:
        return True
    key, left, right = tree[node_idx]
    if not (min_val < key < max_val):
        return False
    return (is_bst(tree, left, min_val, key) and
            is_bst(tree, right, key, max_val))
def check_bst(tree):
    if not tree:
        return "CORRECT"
    return "CORRECT" if is_bst(tree, 0, float("-inf"), float("inf")) else "INCORRECT"
```

●●● Рисунок 1 — Задача 6: Опознание BST

```
$ python3 task6.py
Тест 1 (корректный):      Тест 2 (некорректный):
Узлы: 2 (лев=1, пр=3)     Узлы: 1 (лев=2, пр=3)
DFS: 1<2<3 v             DFS: 2 > 1 — нарушение
output.txt: CORRECT       output.txt: INCORRECT
```

Рисунок 1 — Демонстрация работы: Задача 6 (Опознание BST)

4.2. Задача 9 — Удаление поддеревьев

Для каждого запроса на удаление ключа: рекурсивно считаем размер поддерева `count_subtree`, вычитаем из `node_count`, удаляем узел из словаря. Затем обновляем ссылки родителей (заменяем дочернюю ссылку на 0). После каждого запроса выводим текущий `node_count`.

Листинг 2 — `task9.py`

```
class BST:
    def __init__(self, nodes):
        self.tree = {}
        self.node_count = len(nodes)
        for key, left, right in nodes:
            self.tree[key] = (left, right)
    def count_subtree(self, node):
        if node == 0: return 0
        left, right = self.tree.get(node, (0,0))
        return 1 + self.count_subtree(left) + self.count_subtree(right)
    def delete_subtree(self, key):
        if key not in self.tree: return self.node_count
        self.node_count -= self.count_subtree(key)
        del self.tree[key]
        for parent in self.tree:
            l, r = self.tree[parent]
            if l == key: self.tree[parent] = (0, r)
            elif r == key: self.tree[parent] = (l, 0)
        return self.node_count
```

●●● Рисунок 2 — Задача 9: Удаление поддеревьев

```
$ python3 task9.py
Запросы: 6, 9, 7, 8
Запрос "6": size=1 -> node_count=5
Запрос "9": size=1 -> node_count=4
Запрос "7": не в дереве -> node_count=4
Запрос "8": size=3 -> node_count=1
output.txt: 5 / 4 / 4 / 1
```

Рисунок 2 — Демонстрация работы: Задача 9 (Удаление поддеревьев)

4.3. Задача 13 — Делаю я левый поворот (АВЛ)

Малый левый поворот в корне: правый ребёнок становится новым корнем, его левое поддерево передаётся старому корню как правое. При балансе правого ребёнка = -1 (большой поворот) — сначала правый поворот правого ребёнка.

Листинг 3 — task13.py

```
class AVLTree:
    def left_rotate(self):
        root = self.keys[0]
        right_child = self.tree[root][1]
        if right_child == 0: return self.tree
        right_left, right_right = self.tree.get(right_child, (0,0))
        new_root = right_child
        self.tree[new_root] = (root, right_right)
        self.tree[root] = (self.tree[root][0], right_left)
        del self.tree[right_child]
        return self.tree
```

Рисунок 3 — Задача 13: Левый поворот АВЛ

```
$ python3 task13.py
Корень=-2, правый ребёнок=8
Малый левый поворот:
новый корень = 8
tree[8] = (-2, 9)
tree[-2] = (-7, 3)
output.txt: (дерево с корнем 8)
```

Рисунок 3 — Демонстрация работы: Задача 13 (Левый поворот АВЛ)

4.4*. Задача 14 — Вставка в АВЛ-дерево (дополнительное задание)

Вставка в BST-порядке, затем балансировка корня: если баланс > 1 (левое тяжелее) — правый поворот. Если баланс < -1 — левый поворот. Высота вычисляется рекурсивно.

Листинг 4 — task14.py

```

class AVLTree:
    def insert(self, key):
        current = self.root
        while current in self.tree:
            left, right = self.tree[current]
            if key < current:
                if left == 0: self.tree[current]=(key,right); break
                current = left
            else:
                if right == 0: self.tree[current]=(left,key); break
                current = right
        self.tree[key] = (0, 0)
        self.balance_tree()
    def get_balance(self, node):
        l, r = self.tree.get(node, (0,0))
        return self.get_height(l) - self.get_height(r)

```

●●● Рисунок 4 — Задача 14: Вставка в AVL

```

$ python3 task14.py
Дерево: корень=3, правый=4. Вставка 5:
tree[4]=(0,5) -> баланс корня=-2
Левый поворот: новый корень=4
tree[4]=(3,5), tree[3]=(0,0)
output.txt: корень=4, лев=3, пр=5

```

Рисунок 4 — Демонстрация работы: Задача 14 (Вставка в AVL)

4.5*. Задача 16 — К-й максимум (дополнительное задание)

Структура данных KthMax: max-heap (через `heapq` с отрицанием) + `lookup set` для отслеживания актуальных элементов. `add(x)`: `heappush(-x)` + `lookup.add(x)`. `remove(x)`: `lookup.remove(x)` (ленивое удаление). `kth_max(k)`: извлекаем `k` элементов присутствующих в `lookup`, возвращаем `k`-й.

Листинг 5 — `task16.py`

```

class KthMax:
    def __init__(self):
        self.data = []
        self.lookup = set()
    def add(self, x):
        heapq.heappush(self.data, -x)
        self.lookup.add(x)
    def remove(self, x):
        if x in self.lookup:
            self.lookup.remove(x)
    def kth_max(self, k):
        temp = []
        while len(temp) < k and self.data:
            top = -heapq.heappop(self.data)
            if top in self.lookup: temp.append(top)
        result = temp[-1] if len(temp)==k else None
        for val in temp: heapq.heappush(self.data, -val)
        return result

```

```
$ python3 task16.py
add(5,3,7): lookup={3,5,7}
kth_max(1)=7, kth_max(2)=5, kth_max(3)=3
remove(5): lookup={3,7}
add(10): lookup={3,7,10}
kth_max(1)=10, kth_max(2)=7, kth_max(3)=3
output.txt: 7 / 5 / 3 / 10 / 7 / 3
```

Рисунок 5 — Демонстрация работы: Задача 16 (К-й максимум)

5. Вывод

В ходе лабораторной работы реализованы основные операции над BST и AVL-деревьями: проверка корректности BST (вывод CORRECT/INCORRECT), удаление поддеревьев с подсчётом оставшихся вершин, левый поворот AVL. Дополнительно реализованы вставка в AVL с балансировкой и структура k-го максимума на основе ленивого heap.